

BCH Encoder & Decoder

Bose–Chaudhuri–Hocquenghem Error Correction Codes

Algorithm Reference with Full Worked Example | BCH(15, 5, 3)

Quick Reference — BCH(15, 5, 3)

Parameter	Symbol	Value	Description
Codeword length	n	15 bits	Total transmitted bits
Data bits	k	5 bits	Useful payload
Parity bits	n−k	10 bits	Redundancy added by encoder
Error correction	t	3 errors	Max bit errors correctable
Error detection	2t	6 errors	Max errors detectable (not correctable)
Field order	m	4	$GF(2^4)$ — 16-element field
Code rate	R	$5/15 \approx 33\%$	Efficiency ratio k/n

1. Overview & Motivation

BCH (Bose–Chaudhuri–Hocquenghem) codes are a large family of cyclic error-correcting codes that extend Hamming codes to correct **multiple bit errors**. While Hamming(12,8) corrects only 1 error, BCH codes can be designed to correct any desired number of errors *t* by choosing the appropriate generator polynomial over a Galois Field (GF).

BCH codes are used in: **Flash/SSD storage**, **QR codes**, **satellite communications**, **CD/DVD/Blu-ray**, **deep-space telemetry**, and **5G/LTE wireless** systems.

Hamming vs BCH Comparison

Code	n (bits)	k (bits)	t (correctable)	Overhead	Complexity
Hamming(12,8)	12	8	1	4 bits	Very Low
BCH(15,5,3)	15	5	3	10 bits	Medium
BCH(15,7,2)	15	7	2	8 bits	Medium
BCH(31,21,2)	31	21	2	10 bits	Medium
BCH(63,45,3)	63	45	3	18 bits	Medium
BCH(255,223,4)	255	223	4	32 bits	Higher

■ **Key insight:** BCH codes always satisfy $n = 2^m - 1$ for some integer *m*. BCH(15,5,3) uses *m*=4 because $2^4 - 1 = 15$. The generator polynomial is constructed so it has roots at $\alpha, \alpha^2, \alpha^3, \dots, \alpha^{2t}$ in $GF(2^m)$.

2. Galois Field Arithmetic — GF(2⁴)

BCH codes use arithmetic in **GF(2^m)** — a finite field with 2^m elements. All arithmetic uses **XOR instead of addition** (no carries), and multiplication is done modulo a *primitive polynomial* p(x).

For GF(2⁴), the standard primitive polynomial is:

```
p(x) = x^4 + x + 1    (binary: 10011)

This means: whenever x^4 appears, replace it with (x + 1)
i.e., 10000 XOR 0011 = 0011 mod p(x)
```

GF(2⁴) Element Table (first 8 elements)

Power	Polynomial	4-bit vector	Decimal
0	0	0000	0
α ⁰ = 1	1	0001	1
α ¹	α	0010	2
α ²	α ²	0100	4
α ³	α ³	1000	8
α ⁴	α + 1	0011	3
α ⁵	α ² + α	0110	6
α ⁶	α ³ + α ²	1100	12
α ⁷	α ³ + α + 1	1011	11

■ **GF rule:** α is a primitive element — its powers cycle through all 15 non-zero elements of GF(2⁴). This cycling property is what lets BCH syndromes pinpoint exact error locations.

3. Generator Polynomial g(x)

The generator polynomial g(x) is the LCM of the minimal polynomials of α, α², α³, ..., α^{2t}. For BCH(15,5,3) with t=3, we need roots at α¹ through α⁶:

```
Minimal polynomials over GF(2):
m1(x) = x^4 + x + 1      (roots: α, α^2, α^4, α^8)
m3(x) = x^4 + x^3 + x^2 + x + 1  (roots: α^3, α^6, α^12, α^9)
m5(x) = x^2 + x + 1      (roots: α^5, α^10)

g(x) = LCM(m1, m3, m5)
      = m1(x) × m3(x) × m5(x)

g(x) = x^10 + x^8 + x^5 + x^4 + x^2 + x + 1
      = [1 0 1 0 0 1 1 0 1 1 1] (binary coefficients)
```

■ **Why this works:** Any valid codeword $C(x)$ is a multiple of $g(x)$, so $C(\alpha^i) = 0$ for $i = 1..6$. After transmission, if errors occur, evaluating $R(\alpha^i)$ gives non-zero *syndromes* that encode the error positions.

4. BCH Encoder Algorithm

The encoder uses **systematic encoding** — data bits appear unchanged at the start of the codeword, followed by parity bits. This makes it easy to strip parity at the receiver.

Encoder Block Diagram



Encoder Steps

Step 1: Represent data D as polynomial $D(x)$ of degree $k-1$.

Step 2: Multiply $D(x)$ by x^{n-k} to shift data to high positions.

Step 3: Divide $M(x) = D(x) \cdot x^{n-k}$ by generator $g(x)$ using XOR division.

Step 4: The remainder $R(x)$ is the parity. Codeword $C(x) = M(x) \text{ XOR } R(x)$.

Encoder — Worked Example

Data D = 10110 (5 bits) → Encode to BCH(15,5,3) 15-bit codeword

```

# Input data
D          = 10110
D(x)       = x^4 + x^2 + x

# Step 1 - Shift left by n-k = 10 positions
M(x)       = D(x) * x^10
M          = 101100000000000    (15 bits)

# Step 2 - Generator polynomial
g(x)       = x^10+x^8+x^5+x^4+x^2+x+1
g          = 10100110111

# Step 3 - XOR long division M(x) / g(x)
101100000000000
10100110111          ← XOR with g(x)
■■■■■■■■■■■■■■■■■■
00001110110000
  10100110111          ← XOR with g(x)
  ■■■■■■■■■■■■■■■■
  1110001110000
    10100110111          ← XOR with g(x) ...(continue)

# Step 4 - Remainder R(x) = parity bits
R          = 1101010110    (10 parity bits)

# Step 5 - Codeword = [data | parity]
C          = [10110 | 1101010110]
C          = 101101101010110    (15 bits)
  
```

Codeword Structure — Position Map

b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	1	0	1	1	0	1	0	1	0	1	1	0
D	D	D	D	D	P	P	P	P	P	P	P	P	P	P

■ Blue = Data bit (D) | ■ Yellow = Parity bit (P)

5. BCH Decoder Algorithm

The BCH decoder has **three stages**: compute syndromes → find error-locator polynomial → locate and flip error bits. Each stage uses GF(2^m) arithmetic.

Decoder Pipeline

Stage	Name	Input	Output	Algorithm
1	Syndrome Computation	Received R(x)	S1...S6	Evaluate at α ⁱ
2	Error-Locator Poly	S1...S6	σ(x)	Berlekamp-Massey
3	Root Finding	σ(x)	Error positions	Chien Search
4	Correction	R(x) + errors	C(x) (corrected)	XOR flip bits

Stage 1 — Syndrome Computation

Evaluate the received polynomial R(x) at each root αⁱ of g(x):

```
For i = 1, 2, 3, 4, 5, 6 (i.e., 2t syndromes for t=3):
  S_i = R(alpha^i)
      = r_0 + r_1*(alpha^i) + r_2*(alpha^i)^2 + ... + r_14*(alpha^i)^14

If all S_i == 0 → No errors, output R directly
If any S_i != 0 → Errors detected, proceed to Stage 2
```

■ Syndromes are computed in GF(2⁴) — every multiplication is modulo p(x)=x⁴+x+1, and every addition is XOR. Hardware implementations use LFSR-based syndrome registers.

Stage 2 — Error-Locator Polynomial (Berlekamp-Massey)

Find σ(x) = 1 + σ₁x + σ₂x² + ... + σ_tx^t whose roots are the **inverses** of the error locations.

```
# Berlekamp-Massey Algorithm
Initialize: sigma = [1], B = [1], L = 0, m = 1, b = 1

For n = 1 to 2t:
  # Compute discrepancy delta
  delta = S[n] + sigma[1]*S[n-1] + ... + sigma[L]*S[n-L] (GF arithmetic)

  if delta == 0:
    m += 1 # no update needed
```

```

elif 2*L <= n-1:
    T = sigma.copy()
    sigma = sigma XOR (delta/b) * x^m * B
    L = n - L
    B = T; b = delta; m = 1
else:
    sigma = sigma XOR (delta/b) * x^m * B
    m += 1

# Result: sigma(x) of degree L = number of errors found
    
```

■ **For $t=1$ (single error):** Berlekamp-Massey simplifies to $\sigma(x) = 1 + S_1 \cdot x$, and the Chien search finds one root. This is identical to a Hamming syndrome decode.

Stage 3 — Chien Search (Root Finding)

Test every field element to find roots of $\sigma(x)$. Each root corresponds to an error location:

```

# Chien Search over all n=15 positions
For i = 0 to n-1:
    x_inv = alpha^(-i) = alpha^(n-i)    (since alpha^n = 1)

    # Evaluate sigma at x_inv
    val = sigma_0
        + sigma_1 * (alpha^(-i))^1
        + sigma_2 * (alpha^(-i))^2
        + ...    (all in GF)

    if val == 0:
        ERROR at bit position i  →  flip bit i in R

# After flipping all error bits, output corrected codeword C
# Strip parity: data = C[14:10] (top k=5 bits)
    
```

6. Decoder — Full Worked Example

Scenario: Encode D=10110, transmit 15-bit BCH codeword, inject 2 errors at bit positions 9 and 3. Show the decoder recovering the original data.

Step 1 — Transmitted codeword

C (encoded) = 101101101010110 (from Section 4)
 Bit layout: b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0
 1 0 1 1 0 1 1 0 1 0 1 0 1 1 0

Step 2 — Inject errors at positions 9 and 3

Flip bit 9: 101101101010110
 ↑ b9
 After flip: 101100101010110

 Flip bit 3: 101100101010110
 ↑ b3
 R (received)= 101100101010010 ← 2 bits corrupted

Received Word — Error Positions Highlighted

b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	1	0	0	1	0	1	0	1	0	0	1	0

■ Red = Error positions (b9 and b3)

Step 3 — Syndrome Computation

Evaluate $R(x)$ at α^1 through α^6 in $GF(2^4)$:
 $R(x) = x^{14} + x^{13} + x^{11} + x^{10} + x^8 + x^6 + x^4 + x^1$
 (binary: 101100101010010)

 $S_1 = R(\alpha^1) = \alpha^5$ (non-zero → error!)
 $S_2 = R(\alpha^2) = \alpha^{10}$
 $S_3 = R(\alpha^3) = \alpha^{13}$
 $S_4 = R(\alpha^4) = \alpha^{10}$
 $S_5 = R(\alpha^5) = \alpha^7$
 $S_6 = R(\alpha^6) = \alpha^{11}$

 $S_1 \neq 0 \rightarrow$ Errors detected. Proceeding to Stage 2...

Step 4 — Berlekamp-Massey → Error Locator $\sigma(x)$

Running Berlekamp-Massey with syndromes $S_1..S_6$:

 Iteration 1: $\delta = S_1 = \alpha^5 \rightarrow$ update σ
 Iteration 2: δ computed from $S_1, S_2...$
 ...
 Final result (2 errors found, degree 2):

 $\sigma(x) = 1 + \alpha^{12} * x + \alpha^6 * x^2$
 ↑ σ_0 ↑ σ_1 ↑ σ_2

Step 5 — Chien Search → Locate Errors

Test all 15 positions $i = 0..14$:

```
i=3:  sigma(alpha^(-3)) = sigma(alpha^12)
      = 1 + alpha^12*alpha^12 + alpha^6*alpha^24
      = 1 + alpha^9 + alpha^6    (computing in GF(2^4))
      = ... = 0    ✓  ERROR at position 3!
```

```
i=9:  sigma(alpha^(-9)) = sigma(alpha^6)
      = 1 + alpha^12*alpha^6 + alpha^6*alpha^12
      = 1 + alpha^3 + alpha^3
      = 1 + 0 = ... = 0    ✓  ERROR at position 9!
```

All other positions: $\text{sigma}(\alpha^{-i}) \neq 0 \rightarrow$ no error

Step 6 — Correct and Decode

```
R (received)  = 101100101010010
Flip bit  9   → 101101101010010
Flip bit  3   → 10110110101110
C (corrected) = 10110110101110    ✓  Matches original!
```

```
Extract data (top k=5 bits): C[14:10] = 10110
Decoded D = 10110    ✓  Matches original data!
```

Corrected Codeword — After Decoding

b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	1	0	1	1	0	1	0	1	0	1	1	0

■ Green = Successfully corrected bits (b9 and b3 restored)

7. Algorithm Summary

Encoder Summary

Step	Operation	Formula / Detail
1	Represent data	$D(x)$ = polynomial from k data bits
2	Shift polynomial	$M(x) = D(x) \cdot x^{n-k}$
3	XOR division	$M(x) \div g(x) \rightarrow$ quotient $Q(x)$, remainder $R(x)$
4	Form codeword	$C(x) = M(x) \text{ XOR } R(x)$ [data bits parity bits]

Decoder Summary

Stage	Step	Operation	Key Formula
1	1	Syndrome computation	$S_i = R(\alpha^i)$ for $i=1..2t$
1	2	Check if error-free	All $S_i=0 \rightarrow$ output R as decoded
2	3	Berlekamp-Massey algorithm	Build $\sigma(x)$ of degree = error count
3	4	Chien Search — find roots	$\sigma(\alpha^{-i})=0 \rightarrow$ error at position i
4	5	Bit correction	Flip each error bit in R(x)
4	6	Strip parity	Output $C[n-1:n-k]$ = decoded data

Error Capability Summary

Error Count	BCH(15,5,3) Result	Behaviour
0 errors	✓ Clean	All syndromes zero, pass through
1 error	✓ Corrected	$\sigma(x)$ degree 1, Chien finds 1 root
2 errors	✓ Corrected	$\sigma(x)$ degree 2, Chien finds 2 roots
3 errors	✓ Corrected	$\sigma(x)$ degree 3, Chien finds 3 roots
4 errors	■ Detected only	$\sigma(x)$ may find wrong roots $\rightarrow$ miscorrection
5–6 errors	■ Detected only	Syndromes non-zero but beyond correction
7+ errors	✗ Undetected	May appear as valid codeword

8. Hardware Implementation Notes

Block	FPGA / ASIC Implementation
Encoder	LFSR (Linear Feedback Shift Register) clocked with data bits. Feedback taps match $g(x)$ coefficients. Runs in k clock cycles.

Syndrome unit	2t parallel LFSR circuits, each evaluating $R(\alpha^i)$. Runs in n clock cycles. Results are 2t GF elements.
BM Algorithm	Iterative logic, 2t cycles. Small LUT-based GF multipliers. Outputs $\sigma(x)$ of degree $\leq t$.
Chien Search	n parallel or pipelined GF evaluations. One cycle per position in pipelined form. Outputs error flags.
Total latency	Encode: k+1 cycles. Decode: n + 2t + n cycles $\approx 2n + 2t$ cycles total.

■ **IP cores:** Most FPGA vendors (Xilinx/AMD, Intel/Altera) include parameterisable BCH encoder/decoder IP cores. For custom designs, the open-source **ezpwd** and **bch_verilog** libraries provide synthesisable RTL.

9. Pseudocode — Python Reference

Encoder

```
def bch_encode(data_bits, g_poly, n, k):
    # Shift data left by (n-k) positions
    M = data_bits << (n - k)

    # XOR polynomial long division
    for i in range(k - 1, -1, -1):
        if (M >> (i + n - k)) & 1:
            M ^= (g_poly << i)

    parity = M & ((1 << (n - k)) - 1)
    codeword = (data_bits << (n - k)) | parity
    return codeword
```

Syndrome Computation

```
def compute_syndromes(received, alpha_powers, n, t, gf_mult, gf_add):
    syndromes = []
    for i in range(1, 2*t + 1):
        s = 0
        for j in range(n):
            if (received >> j) & 1:
                s = gf_add(s, alpha_powers[(i * j) % (n)]) # alpha^(i*j)
        syndromes.append(s)
    return syndromes # list of 2t GF elements
```

Berlekamp-Massey (simplified)

```
def berlekamp_massey(syndromes, t, gf_mult, gf_div, gf_add):
    sigma, B = [1], [1]
    L, m, b = 0, 1, 1
    for n in range(1, 2*t + 1):
        delta = syndromes[n-1]
        for i in range(1, L + 1):
            delta = gf_add(delta, gf_mult(sigma[i], syndromes[n-1-i]))
        if delta == 0:
            m += 1
        elif 2*L <= n - 1:
            T = sigma[:]
```

```
        coeff = gf_div(delta, b)
        sigma = poly_xor(sigma, poly_scale_shift(B, coeff, m))
        L, B, b, m = n - L, T, delta, 1
    else:
        sigma = poly_xor(sigma, poly_scale_shift(B, gf_div(delta,b), m))
        m += 1
    return sigma    # error locator polynomial
```

Chien Search & Correction

```
def chien_search_and_correct(received, sigma, n, alpha_inv, gf_mult, gf_add):
    corrected = received
    for i in range(n):
        # Evaluate sigma at alpha^(-i)
        val = 0
        for j, coeff in enumerate(sigma):
            val = gf_add(val, gf_mult(coeff, alpha_inv[i*j % n]))
        if val == 0:
            corrected ^= (1 << i)    # flip error bit
    return corrected
```

10. Key Terms Glossary

Term	Definition
$GF(2^m)$	Galois Field with 2^m elements. All BCH arithmetic is performed here.
α (alpha)	Primitive element of $GF(2^m)$. Its powers generate all non-zero field elements.
$g(x)$	Generator polynomial. Every valid codeword is a multiple of $g(x)$.
Syndrome S_i	Value $R(\alpha^i)$. Non-zero syndromes indicate the presence of errors.
$\sigma(x)$	Error-locator polynomial. Its roots α^{-i} identify error positions i .
Berlekamp-Massey	Efficient iterative algorithm to compute $\sigma(x)$ from syndromes in $O(t^2)$ time.
Chien Search	Algorithm to find all roots of $\sigma(x)$ by evaluating at every field element.
Systematic code	Encoding where data bits appear unchanged in the codeword (simplifies stripping parity).
n, k, t	BCH parameters: n = codeword length, k = data bits, t = errors correctable.
Code rate R	$R = k/n$. Fraction of bits that carry data. Higher R = more efficient, but less protection.

BCH codes represent a fundamental advance over Hamming — the same systematic, polynomial-based framework but extended to $GF(2^m)$ arithmetic enables correction of arbitrary numbers of errors. The three-stage decode pipeline (syndromes → Berlekamp-Massey → Chien search) is the foundation of modern storage and communication error correction.